

WebView 在项目中的使用及优化建议

WebView 在项目中的使用详解与优化建议

一、WebView 的核心实现

项目中 WebView 功能通过 “自定义封装类 + 基础控制器” 两层结构实现，覆盖不同场景的网页加载需求：

1. XMWebView - 自定义 WebView 封装类

XMWebView 是项目中 WebView 的核心封装，优势在于 **同时支持 UIWebView 与 WKWebView**，可灵活适配不同 iOS 系统版本，核心功能如下：

核心功能	说明
多方式加载	支持加载 URL、URL 字符串、HTML 字符串、- NSURLRequest，满足不同网页源需求
统一导航控制	封装前进、后退、刷新、停止加载等方法，无需重复实现
统一代理回调	通过 XMWebViewDelegate 统一 UIWebView 与 WKWebView 的代理方法，降低适配成本
加载进度显示	通过 KVO 监听 WKWebView 的 estimatedProgress 属性，实现进度条实时更新
外部应用调用拦截	拦截网页中的特殊链接（如拨打电话、打开第三方 App），触发原生逻辑

2. ZYWebBaseViewController - 网页显示基础控制器

ZYWebBaseViewController 是项目中实际用于展示网页的控制器，基于 UIWebView 实现，提供完整的页面交互能力，主要特性如下：

- **多文件类型支持**：可加载普通网页（HTML）、TXT 文档、PDF 文档，自动适配文档格式；
- **加载状态提示**：集成 MBProgressHUD，显示“加载中”“加载失败”等提示，提升用户感知；
- **标题自动同步**：从网页的 `<title>` 标签中获取标题，自动设置为控制器导航栏标题；
- **原生交互能力**：通过 URL Scheme 拦截实现 Web 端与原生功能的通信（如跳转原生页面、调用原生方法）。

二、WebView 与原生的交互方式

项目中 WebView 与原生的交互核心为 **URL Scheme 拦截**，这是一种轻量、易实现的交互方案，流程如下：

1. **Web 端触发**：Web 页面通过跳转特殊格式的 URL（如 `contract://open?contractId=123、login://jump`）触发交互需求；
2. **原生端拦截**：在 WebView 的 `shouldStartLoadWithRequest` 代理方法中，拦截上述特殊 URL；
3. **执行原生逻辑**：解析 URL 的 Scheme（如 `contract、login`）与参数（如 `contractId`），执行对应原生功能（如打开合同详情页、跳转登录页）。

三、代码优化建议

通过分析现有实现，从“性能、兼容性、用户体验”角度提出以下优化方向，附具体代码示例：

1. 全面迁移到 WKWebView

（1）问题

当前 `ZYWebBaseViewController` 依赖已弃用的 `UIWebView`（iOS 12+ 官方弃用），而 `XMWebView` 虽支持 `WKWebView` 但未充分利用，导致性能差、内存占用高、不支持现代 Web 特性。

（2）优化理由

WKWebView 相比 UIWebView：

- 性能提升：采用独立进程渲染，不阻塞主线程；
- 内存占用：比 UIWebView 低约 50%；

- 功能支持：支持 JavaScript ES6+、Service Worker、WebGL 等现代特性。

(3) 核心迁移代码

```
// ZYWebBaseViewController 中替换为WKWebView
- (void)setupWebView {
    // 1. 初始化WKWebView配置
    WKWebViewConfiguration *config = [[WKWebViewConfiguration alloc] init];
    // 启用JavaScript（根据需求配置）
    config.preferences.javaScriptEnabled = YES;

    // 2. 创建WKWebView
    self.webView = [[WKWebView alloc] initWithFrame:self.view.bounds configuration:config];
    self.webView.navigationDelegate = self; // 设置导航代理
    self.webView.uiDelegate = self;       // 设置UI代理
    [self.view addSubview:self.webView];

    // 3. 监听加载进度（用于显示进度条）
    [self.webView addObserver:self forKeyPath:@"estimatedProgress" options:
    NSKeyValueObservingOptionNew context:nil];
}

// 实现WKWebView导航代理（替代UIWebView代理方法）
- (void)webView:(WKWebView *)webView didFinishNavigation:(WKNavigation *)navigation {
    // 加载完成：隐藏HUD、同步标题
    [self endMBProgressHUD];
    // 获取网页标题并设置为控制器标题
    [webView evaluateJavaScript:@"document.title" completionHandler:^(id _Nullable result,
    NSError * _Nullable error) {
        if (result && !error) {
            self.title = result;
        }
    }];
}
```

2. 资源加载优化

(1) 问题

现有实现无资源缓存策略，重复加载相同网页 / 资源，导致加载慢、流量消耗大。

(2) 优化建议

- **启用 WKWebView 默认缓存：**配置 `WKWebsiteDataStore` 启用磁盘缓存；
- **预加载关键资源：**对高频访问的网页（如帮助中心），在空闲时预加载；
- **资源压缩：**与 Web 端协商，启用 Gzip 压缩 HTML/CSS/JS 资源。

(3) 核心优化代码

```
// 1. 配置WKWebView缓存（长期缓存）
- (void)setupWebViewCache {
    WKWebViewConfiguration *config = [[WKWebViewConfiguration alloc] init];
    // 使用持久化数据存储（支持磁盘缓存）
    config.websiteDataStore = [WKWebsiteDataStore defaultDataStore];

    // 2. 设置缓存策略（仅在无网络时使用缓存，根据需求调整）
    self.webView.configuration.websiteDataStore = config.websiteDataStore;
}

// 2. 预加载关键网页（如App启动后空闲时）
- (void)preloadKeyWebPage {
    // 仅在WiFi环境下预加载（避免消耗用户流量）
    if ([self isWiFiConnected]) {
        NSURL *preloadURL = [NSURL URLWithString:@"https://example.com/help"];
        NSURLRequest *request = [NSURLRequest requestWithURL:preloadURL];
        [self.webView loadRequest:request];
    }
}

// 辅助方法：判断是否为WiFi环境
- (BOOL)isWiFiConnected {
    NSArray *interfaces = [[NSNetworkActivityIndicatorManager sharedManager]
networkInterfaces];
    for (NSString *interface in interfaces) {
        if ([interface isEqualToString:@"en0"]) {
            NSDictionary *info = [[NSNetworkActivityIndicatorManager sharedManager]
interfaceInfoForInterface:interface];
            return [[info objectForKey:@"ssid"] length] > 0;
        }
    }
}
```

```
}  
return NO;  
}
```

3. 内存管理优化

(1) 问题

现有实现未妥善处理 WebView 的生命周期，可能导致内存泄漏（如未移除 KVO 监听、代理未置空）。

(2) 优化建议

- **移除 KVO 监听**：在控制器销毁时，移除对 WKWebView `estimatedProgress` 的监听；
- **置空代理与 WebView**：控制器销毁前，将 WebView 的代理置空并移除引用；
- **清理 WebView 数据**：按需清理缓存数据（如用户退出登录时）。

(3) 核心优化代码

```
- (void)dealloc {  
    // 1. 移除KVO监听（避免野指针）  
    if (self.webView) {  
        [self.webView removeObserver:self forKeyPath:@"estimatedProgress"];  
    }  
  
    // 2. 置空代理（避免循环引用）  
    self.webView.navigationDelegate = nil;  
    self.webView.uiDelegate = nil;  
  
    // 3. 移除WebView引用（触发内存释放）  
    [self.webView removeFromSuperview];  
    self.webView = nil;  
  
    // 4. 清理WebView缓存数据（按需执行，如退出登录时）  
    [self clearWebViewCache];  
}  
// 清理WebView缓存（包括Cookie、本地存储等）
```

```
- (void)clearWebViewCache {
    NSMutableSet *websiteDataTypes = [NSMutableSet setWithObjects:
        WKWebsiteDataTypeDiskCache,
        WKWebsiteDataTypeMemoryCache,
        WKWebsiteDataTypeCookies,
        nil];
    NSDate *dateFrom = [NSDate dateWithTimeIntervalSince1970:0];
    [[WKWebsiteDataStore defaultDataStore] removeDataOfTypes:websiteDataTypes
    modifiedSince:dateFrom completionHandler:^(
        // 清理完成回调
    )];
}
```

4. 错误处理与用户体验优化

(1) 问题

现有错误处理仅显示简单提示，未区分错误类型（如无网络、超时、404），用户无法明确问题原因，也无重试入口。

(2) 优化建议

- **分类错误处理**：根据错误码区分“无网络”“加载超时”“服务器错误”等，显示对应提示页面；
- **添加重试入口**：错误页面提供“重试”按钮，支持重新加载；
- **显示加载进度条**：通过 WKWebView 的 `estimatedProgress` 实现进度条，提升加载感知。

(3) 核心优化代码

```
// 1. 增强错误处理（WKWebView导航代理方法）
- (void)webView:(WKWebView *)webView didFailNavigation:(WKNavigation *)navigation
withError:(NSError *)error {
    [self endMBProgressHUD];

    // 过滤“页面跳转中断”等非实际错误
    if (error.code == NSURLErrorCancelled) {
        return;
    }
}
```

```

// 根据错误类型显示对应页面
if (error.code == NSErrorNotConnectedToInternet) {
    [self showNoNetworkView]; // 无网络页面
} else if (error.code == NSErrorTimedOut) {
    [self showTimeoutView]; // 加载超时页面
} else if (error.code >= 400 && error.code < 500) {
    [self show404View]; // 404页面
} else {
    [self showErrorViewWithMessage:error.localizedDescription]; // 通用错误页面
}
}

// 2. 添加加载进度条 (viewDidLoad中初始化)
- (void)setupProgressView {
    // 进度条位置：导航栏下方 (高度2px)
    self.progressBar = [[UIProgressView alloc] initWithFrame:CGRectMake(0, k_Height_NavBar, self.view.frame.size.width, 2)];
    self.progressBar.tintColor = [UIColor blueColor]; // 进度条颜色
    self.progressBar.trackTintColor = [UIColor clearColor]; // 背景色透明
    [self.view addSubview:self.progressBar];
}

// 3. KVO监听进度更新
- (void)observeValueForKeyPath:(NSString *)keyPath ofObject:(id)object change:(
NSDictionary<NSKeyValueChangeKey,id> *)change context:(void *)context {
    if ([keyPath isEqualToString:@"estimatedProgress"] && object == self.webView) {
        CGFloat progress = [change[NSKeyValueChangeNewKey] floatValue];
        self.progressBar.progress = progress;

        // 进度达到100%后，0.3秒内隐藏进度条 (避免突兀消失)
        if (progress >= 1.0) {
            dispatch_after(dispatch_time(DISPATCH_TIME_NOW, (int64_t)(0.3 * NSEC_PER_SEC)),
dispatch_get_main_queue(), ^{
                self.progressBar.hidden = YES;
            });
        } else {
            self.progressBar.hidden = NO; // 有进度时显示
        }
    }
}

// 4. 无网络页面 (示例：显示图片+重试按钮)
- (void)showNoNetworkView {

```

```
// 移除原有错误页面（避免叠加）
[self.noNetworkView removeFromSuperview];

// 创建无网络页面
self.noNetworkView = [[UIView alloc] initWithFrame:self.view.bounds];
self.noNetworkView.backgroundColor = [UIColor whiteColor];

// 添加提示图片
UIImageView *imgView = [[UIImageView alloc] initWithImage:[UIImage imageNamed:@"no_network"]];
imgView.center = self.noNetworkView.center;
[self.noNetworkView addSubview:imgView];

// 添加重试按钮
UIButton *retryBtn = [UIButton buttonWithType:UIButtonTypeSystem];
retryBtn.frame = CGRectMake(0, CGRectGetMaxY(imgView.frame) + 20, 100, 40);
retryBtn.centerX = self.noNetworkView.centerX;
[retryBtn setTitle:@"重新加载" forState:UIControlStateNormal];
[retryBtn addTarget:self action:@selector(reloadWebPage) forControlEvents:
UIControlEventTouchUpInside];
[self.noNetworkView addSubview:retryBtn];

[self.view addSubview:self.noNetworkView];
}

// 重试加载方法
- (void)reloadWebPage {
    [self.noNetworkView removeFromSuperview]; // 移除错误页面
    [self.webView reload]; // 重新加载网页
    [self startMBProgressHUD]; // 显示加载HUD
}
```

四、总结

项目中 WebView 实现已覆盖基础网页加载与交互需求，但需从“兼容性、性能、用户体验”三方面重点优化，核心优化方向如下：

1. **全面迁移到 WKWebView**：替代弃用的 UIWebView，提升性能与现代 Web 特性支持；
2. **优化资源加载**：启用缓存、预加载关键资源、压缩传输，减少加载时间与流量消耗；
3. **加强内存管理**：移除 KVO 监听、置空代理、清理缓存，避免内存泄漏；

4. **优化错误处理**：分类错误类型、提供重试入口、显示进度条，提升用户体验。

通过上述优化，可显著提升 WebView 的稳定性与流畅度，让网页功能更贴近原生体验。

（注：文档部分内容可能由 AI 生成）